

AgentKart

Production-Ready Product & Engineering Plan

Global AI Agent Marketplace

Core Product Promise

A business describes the work it needs done. AgentKart helps it discover the right AI agent, evaluate it, hire or subscribe to it, and run the work reliably.

BUSINESS NEED -> DISCOVER -> MATCH -> TEST -> HIRE -> PAY -> DEPLOY -> RUN -> MONITOR -> SCALE / REPLACE

Document purpose: define the Day-1 production architecture, product scope, implementation sequence, security model, data model, operational controls, and scaling path for AgentKart. This plan intentionally keeps the full enterprise architecture from the supplied implementation blueprint while assigning clear responsibilities to each subsystem.

1. Executive Summary

AgentKart is a B2B marketplace and execution platform for AI agents. The product is not only a directory: the core experience is a business-to-agent transaction loop in which a founder states a business problem, discovers suitable agents, compares capabilities and economics, hires or buys access, and then runs the agent through a reliable execution layer.

The engineering target is production-grade architecture from Day 1. The supplied enterprise blueprint calls for durable execution, hard multi-tenancy, cloud-native deployment, dynamic model routing, PostgreSQL, Neo4j, vector storage, Redis, FastAPI, a TypeScript core API, Next.js, Docker, Kubernetes, and Terraform. This plan adopts that direction and maps it to AgentKart.

Architecture decision

Use Next.js + TypeScript for the frontend, NestJS + TypeScript for the core platform API, FastAPI + Python for AI/ML services, Temporal for durable workflows, PostgreSQL as transactional source of truth, Neo4j for relationship data, Qdrant for vector retrieval, Redis for ephemeral/high-speed state, S3-compatible object storage for artifacts, and Kubernetes on AWS or GCP for production workloads.

2. Product Definition

2.1 Product goal

Make AI capability purchasable and operable like business talent: discover it, evaluate it, hire it, use it, monitor it, pay for it, and replace it when it no longer fits.

2.2 Primary users

User	Primary jobs	Key surfaces
Founder / buyer	Describe need, search, compare, test, hire, monitor, pay	Marketplace, requirement intake, agent detail, checkout, workspace
Agent provider / developer	Publish an agent, define skills/tools, pricing, versions, availability	Provider console, agent publishing, versions, analytics
Enterprise admin	Manage organization, members, policies, budgets, integrations	Admin workspace, RBAC, audit, billing
Platform operator	Moderate listings, support customers, observe health, handle disputes	Internal admin, moderation, operations dashboard

3. Product Lifecycle

1. DISCOVER -> 2. DESCRIBE NEED -> 3. MATCH -> 4. EVALUATE -> 5. HIRE
6. PAYMENT -> 7. PROVISION -> 8. EXECUTE -> 9. MONITOR -> 10. BILL
11. REVIEW -> 12. SCALE / REPLACE

3.1 Founder journey

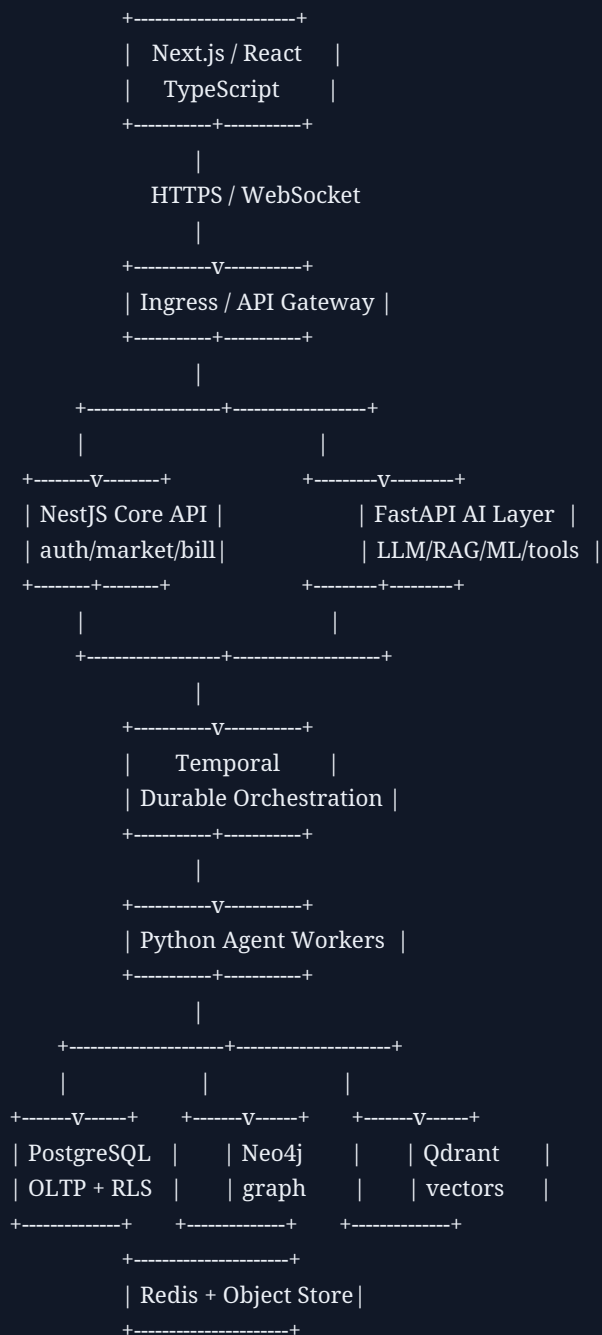
1. Create an account and organization workspace.
2. Describe the required outcome in natural language.
3. Receive a ranked set of agents based on semantic fit, skills, integrations, price, reliability, and policy constraints.
4. Open agent profiles, inspect capabilities, pricing, supported tools, permissions, reviews, and test results.
5. Run a controlled trial or sandbox task where supported.
6. Choose purchase/subscription terms and complete checkout.
7. Provision the agent into the organization workspace with scoped permissions.
8. Run tasks and stream status/results to the user.
9. Inspect usage, outcomes, cost, errors, and audit events.
10. Review, scale, pause, replace, or retire the agent.

3.2 Provider journey

11. Create provider account and organization.
12. Define an agent profile: purpose, skills, supported inputs/outputs, tools, limits, pricing, runtime requirements.

13. Publish a versioned listing for review.
14. Pass automated validation and security checks.
15. Publish to marketplace.
16. Receive purchases/hiring events and provisioned executions.
17. Monitor agent performance and customer outcomes.
18. Publish new versions with controlled rollout and rollback.

4. Target System Architecture



Docker -> Kubernetes -> AWS / GCP

Responsibility rule: PostgreSQL remains the source of truth for accounts, organizations, commercial records, subscriptions, permissions, billing references, and operational metadata. Neo4j holds relationship-heavy graph representations. Qdrant serves semantic retrieval. Redis handles cache, rate limiting, pub/sub, and transient context. Temporal owns durable workflow state. Kubernetes is the runtime control plane for containerized workloads.

5. Technology Stack - Locked Baseline

Layer	Technology	Why it belongs
Web app	Next.js + React + TypeScript	SSR/marketing pages plus authenticated application dashboards; strong type safety across frontend contracts.
UI	Tailwind CSS + shadcn/ui	Consistent, maintainable design system for marketplace and dashboard surfaces.
Core backend	NestJS + TypeScript	Structured business API for auth, organizations, marketplace, billing, orders, permissions, CRUD, admin.
AI backend	FastAPI + Python	Native AI/ML ecosystem for agents, LLM calls, RAG, embeddings, model routing, tool execution.
Workflow	Temporal	Durable execution, retries, timeouts, state recovery, long-running workflows.
Relational DB	PostgreSQL	Transactional source of truth, strong consistency, RLS, mature migrations/backups.
Graph DB	Neo4j	Relationship and permission graph between organizations, agents, skills, tools, integrations.
Vector DB	Qdrant	Semantic search, agent matching, memory and retrieval with tenant-aware filtering.
Cache / realtime infra	Redis	Caching, rate limiting, short-lived state, pub/sub, stream coordination.
Object storage	S3-compatible storage	Agent artifacts, files, evaluation data, reports, logs requiring long-term retention.
Containers	Docker	Reproducible environments across development, CI, and production.

Layer	Technology	Why it belongs
Orchestration	Kubernetes (EKS/GKE)	Production scheduling, health management, autoscaling, workload isolation.
IaC	Terraform	Repeatable cloud/infrastructure provisioning.
Observability	OpenTelemetry + Prometheus + Grafana + Sentry	Metrics, traces, dashboards, error tracking, SLOs.
Payments	Stripe or regional equivalent	Subscriptions, metered billing, payouts and marketplace economics.

6. Backend Service Boundaries

6.1 NestJS core modules

- Identity and organizations
- Users, membership, roles and permissions
- Agent catalog and listings
- Search/filter API aggregation
- Orders, subscriptions and invoices
- Payments and provider payouts
- Reviews and reputation
- Notifications and messaging
- Admin/moderation
- Audit events and policy records

6.2 FastAPI AI modules

- Agent execution API
- Requirement understanding and structured extraction
- Embedding and retrieval pipeline
- Agent matching / ranking
- LLM gateway and provider abstraction
- Tool execution adapters
- RAG and memory services
- Prompt / model configuration
- Evaluation and scoring
- AI safety and output validation

6.3 Temporal workflows

- AgentTaskWorkflow: execute a business task end-to-end
- ProvisionAgentWorkflow: prepare runtime configuration and permissions
- PaymentProvisioningWorkflow: coordinate payment and entitlement events
- EvaluationWorkflow: run standardized benchmark/test tasks

- AgentVersionRolloutWorkflow: staged deployment and rollback
- UsageBillingWorkflow: collect execution metrics for metering

7. API and Event Design

7.1 External API style

Use versioned REST APIs for the majority of business operations and WebSockets for realtime execution status. Keep AI-heavy execution behind FastAPI and keep commerce/identity behind NestJS. Avoid exposing internal service credentials to the browser.

7.2 Representative endpoints

Method	Endpoint	Purpose
POST	/api/v1/requirements	Create a business requirement.
GET	/api/v1/agents	Search and filter marketplace agents.
GET	/api/v1/agents/{id}	Retrieve agent profile and commercial metadata.
POST	/api/v1/agents/{id}/trial	Start a controlled trial.
POST	/api/v1/orders	Create an order / hiring request.
POST	/api/v1/subscriptions	Create or modify subscription.
POST	/api/v1/executions	Start an agent execution.
GET	/api/v1/executions/{id}	Retrieve execution status and results.
POST	/api/v1/reviews	Submit review after eligible usage.
GET	/api/v1/usage	Retrieve usage/cost metrics.

7.3 Internal events

```
RequirementCreated
AgentMatched
TrialStarted
OrderCreated
PaymentSucceeded
AgentProvisioned
ExecutionStarted
ExecutionCompleted
ExecutionFailed
UsageRecorded
InvoiceGenerated
AgentVersionPublished
AgentSuspended
```

8. Data Model

8.1 Core PostgreSQL entities

Entity	Important fields / notes
users	id, email, identity_provider_id, status, timestamps
organizations	id, name, plan, status, created_at
organization_members	organization_id, user_id, role, status
agents	id, provider_org_id, name, description, category, status, current_version_id
agent_versions	agent_id, version, runtime_spec, skill_spec, pricing_ref, security_state
skills	id, name, taxonomy_id, metadata
agent_skills	agent_id/version_id, skill_id, proficiency/evidence
tools	id, provider, capability, security_class
agent_tools	agent_version_id, tool_id, permission_scope
requirements	id, organization_id, raw_text, structured_intent, embedding_ref
matches	requirement_id, agent_id, score, explanation_ref, created_at
orders	id, buyer_org_id, agent_id, status, commercial_terms
subscriptions	id, order_id, plan, status, limits
executions	id, organization_id, agent_id, workflow_id, status, cost
usage_records	execution_id, metric, quantity, unit_price
reviews	order_id, reviewer_id, agent_id, rating, text, moderation_state
audit_events	tenant_id, actor_id, action, resource_type, resource_id, metadata, timestamp

8.2 Neo4j graph

Use Neo4j only for relationship queries that benefit from graph traversal. Keep canonical commercial and identity records in PostgreSQL and synchronize graph projections deliberately.

```
(:Organization)-[:MEMBER]->(:User)
(:Organization)-[:HIRES]->(:Agent)
(:Agent)-[:HAS_SKILL]->(:Skill)
(:Agent)-[:USES_TOOL]->(:Tool)
(:Tool)-[:INTEGRATES_WITH]->(:Service)
(:Organization)-[:ALLOWS]->(:Integration)
(:Agent)-[:DEPENDS_ON]->(:Capability)
```

8.3 Qdrant collections

- `agent_profiles`: semantic representations of capabilities and supported use cases
- `requirements`: reusable requirement embeddings for matching and analytics
- `agent_memory`: tenant-scoped long-term semantic memory where appropriate
- `evaluation_corpus`: embeddings for benchmark/example retrieval

Tenant isolation rule for vector data

Every vector query must apply a `tenant_id` / access-scope filter before semantic ranking. Never rely on the vector score alone to establish authorization.

9. Multi-Tenancy and Security

Treat every customer organization as an isolated tenant. The supplied blueprint specifically calls for hard multi-tenancy, PostgreSQL Row-Level Security, strict tenant metadata filtering in vector storage, and OAuth tokens bound to the tenant execution context.

Control	Design
Tenant identity	Every authenticated request resolves to user + organization + role context.
PostgreSQL RLS	Tenant-owned tables enforce <code>organization_id</code> policies at the database layer.
Application authorization	NestJS guards/policies enforce RBAC/ABAC before business actions.
Neo4j isolation	Tenant / organization IDs are included in graph relationships and traversal predicates.
Qdrant isolation	Strict payload filters applied before similarity search results are accepted.
Secrets	Use managed secrets/KMS; never store provider tokens in application tables unencrypted.
OAuth	Store refresh/access tokens encrypted and bind credentials to organization + integration identity.
Auditability	Record security-sensitive mutations and agent actions with immutable-style audit events.
Network	Private networking for databases and internal services; TLS at all external boundaries.
Least privilege	Agents receive only the tools and scopes required by the job.

10. Agent Marketplace Quality Model

Marketplace trust is a product feature. An agent listing should be evaluated using a combination of structured metadata, benchmark performance, reliability, cost, permissions, version history, and verified customer feedback.

Dimension	Example signals
Capability fit	Skill match, input/output compatibility, supported domains, tool coverage
Quality	Benchmark accuracy, task completion rate, human evaluation
Reliability	Success rate, p95 execution time, retry/failure rate
Economics	Subscription price, token cost, tool cost, total cost per completed task
Security	Permission scope, integration requirements, security review status
Experience	Rating, reviews, dispute rate, onboarding friction
Recency	Agent version freshness, model freshness, benchmark recency

11. Matching Engine

The matching engine should be a ranking pipeline, not a single vector lookup.

Requirement text

- > structured intent extraction
- > embedding
- > Qdrant candidate retrieval
- > hard filters (category, tools, policy, budget, geography if required)
- > Neo4j relationship / permission checks
- > feature scoring
- > optional learned reranker
- > explanation + ranked agents

11.1 Initial scoring features

- Semantic similarity
- Required-skill coverage
- Required-tool / integration coverage
- Budget fit
- Historical success rate
- Latency/reliability
- Customer rating confidence

- Security/policy compatibility
- Provider trust / verification

11.2 Explainability

Every match should have a compact explanation such as: “Matched because it supports Shopify + Zendesk, has 94% verified task success, fits your budget, and is approved for your organization.” Never expose internal chain-of-thought; provide outcome-relevant factors only.

12. Agent Execution Lifecycle

```
User request
-> Auth + tenant validation
-> Create execution record
-> Start Temporal workflow
-> Resolve agent version
-> Load skills / config
-> Load permitted tools
-> Retrieve context
-> Select model(s)
-> Execute tool-calling loop
-> Validate outputs
-> Persist result + usage
-> Stream status/result
-> Finish / retry / compensate
```

The supplied blueprint uses the same durable request lifecycle: authentication and tenant validation at ingestion, Temporal orchestration, Python worker execution, graph/vector context retrieval, model routing, and realtime streaming.

13. Reliability and Failure Handling

Failure	Required behavior
LLM timeout	Retry according to policy; fallback model/provider where safe.
LLM provider outage	Shift traffic to configured fallback provider; surface degraded state.
Worker crash	Temporal resumes workflow from durable state.
Duplicate request	Idempotency key prevents duplicate order/execution effects.
Tool/API failure	Retry with backoff; circuit-break repeated failures; preserve audit trail.
Payment success / provisioning failure	Use Temporal compensation workflow; do not silently lose entitlement state.

Failure	Required behavior
Database transient outage	Use bounded retries and health-aware connection management.
Bad agent version	Pause version, rollback to last-known-good version.
Runaway agent loop	Loop counter / execution budget / token and time limits terminate safely.

14. Cost and Billing Model

Design commercial metering so the platform can support multiple pricing models without changing its execution architecture.

- Subscription pricing per agent
- Usage pricing per task / execution
- Token-based metering
- Tool-call based metering
- Compute-time metering
- Hybrid subscription + usage
- Provider revenue share / platform commission

Every execution should emit immutable usage facts such as model tokens, duration, tool calls, provider costs, and billable units. Pricing calculations happen from recorded usage facts rather than directly from transient runtime state.

15. Kubernetes and Cloud Architecture

```

AWS or GCP
|
+-- Managed Kubernetes (EKS/GKE)
|   +-- ingress / edge
|   +-- NestJS pods
|   +-- FastAPI pods
|   +-- Temporal workers
|   +-- realtime gateway
|   +-- background workers
|
+-- Managed PostgreSQL
+-- Redis
+-- Neo4j (managed/self-managed based on ops decision)
+-- Qdrant
+-- S3 object storage
+-- KMS / Secrets Manager
+-- Monitoring / logging

```

15.1 Kubernetes workload principles

- Separate deployments for core API and AI services
- Dedicated worker deployments for different execution classes
- Horizontal Pod Autoscaler driven by CPU plus application metrics/queue depth
- Readiness/liveness/startup probes
- Resource requests/limits
- Pod Disruption Budgets for critical services
- Rolling deployments with automatic rollback conditions
- Namespace and network-policy separation
- Dedicated service accounts and cloud identities

16. CI/CD and Infrastructure as Code

Terraform should provision cloud resources and Kubernetes dependencies in a repeatable way.

Application CI should build immutable container images, run tests/security checks, publish artifacts, then deploy through staged environments.

```
Developer commit
-> lint / typecheck / unit tests
-> integration tests
-> security + dependency scanning
-> container build
-> image registry
-> staging deployment
-> smoke / contract tests
-> approval or policy gate
-> production rollout
-> health checks
-> monitor / rollback
```

17. Testing Strategy

Level	Coverage
Unit	Business rules, scoring functions, authorization policies, serializers, cost calculations
Integration	PostgreSQL, Redis, Neo4j, Qdrant, Temporal test environment, payment provider sandbox
Contract	NestJS <-> FastAPI payloads, WebSocket events, provider integrations
End-to-end	Founder journey from requirement to trial/hire/execution/result
AI evaluation	Task suites, golden datasets, regression checks, hallucination/grounding tests

Level	Coverage
Load	API throughput, vector search, concurrent executions, WebSocket fanout
Resilience	Kill worker mid-task, provider outage, DB failover simulation, duplicate events
Security	SAST/DAST, dependency scanning, authz tests, tenant isolation tests, secret scanning

18. Observability and SLOs

The supplied blueprint recommends Prometheus/Grafana for worker scaling signals and model-failover monitoring. AgentKart should extend that into customer-facing operational SLOs.

Metric	Initial production target / rule
API availability	>= 99.9% monthly for core marketplace APIs
Execution start latency	Track p50/p95 from accepted request to workflow start
Agent success rate	Track by agent, version, provider, model and task type
LLM fallback rate	Alert on sustained increase above baseline
Vector search latency	Track p50/p95 and saturation
Temporal workflow failures	Alert by workflow class and failure reason
Cost per successful task	Dashboard by agent/provider/customer segment
Tenant isolation violations	Any confirmed event is Sev-0/critical and triggers incident response
Frontend errors	Track JS exceptions and user-impacting API failures via Sentry/OTel

19. AI Safety and Guardrails

- Explicit tool allowlists per agent version
- Credential scopes restricted to tenant and approved integration
- Maximum execution time
- Maximum step / loop count
- Maximum token budget
- Structured output schemas
- Input/output validation
- Sensitive-action approval gates where necessary
- Prompt injection defenses for retrieved/tool content
- Audit trail of tool calls and external side effects
- Kill switch to suspend an agent/version globally or per tenant

20. Administrative and Moderation System

- Agent listing review and approval
- Provider verification
- Content moderation
- Security review status
- Agent version rollback
- Customer dispute management
- Refund / suspension workflows
- Global and tenant-specific kill switches
- Usage/cost anomaly detection
- Audit-log inspection

21. Delivery Roadmap

Phase 0 - Foundation (Weeks 1-2)

- Finalize domain model and service boundaries
- Set up repositories, branching, code standards, CI
- Provision non-production cloud accounts/projects
- Terraform baseline
- Kubernetes baseline cluster
- Managed PostgreSQL, Redis, object storage
- Observability baseline
- Authentication and organization model

Phase 1 - Marketplace Core (Weeks 3-7)

- Agent provider onboarding
- Agent listing model/versioning
- Marketplace search/filter
- Agent profile pages
- Requirement intake
- Order and subscription model
- Payments sandbox
- Founder and provider dashboards
- Initial audit logging

Phase 2 - AI Matching (Weeks 8-11)

- Requirement structuring
- Embedding pipeline
- Qdrant retrieval
- Neo4j relationship graph
- Ranking model
- Match explanations
- Agent trial framework

- Basic evaluation suite

Phase 3 - Durable Execution (Weeks 12-16)

- Temporal cluster integration
- AgentTaskWorkflow
- Python workers
- Tool execution framework
- Execution state persistence
- Realtime execution updates
- Retry/fallback policies
- Usage metering

Phase 4 - Production Hardening (Weeks 17-22)

- RLS and tenant-isolation verification
- Load testing
- Resilience testing
- Security hardening
- Backup and disaster recovery drills
- Kubernetes autoscaling
- SLO dashboards
- Runbooks and incident response
- Production launch checklist

Phase 5 - Scale and Intelligence (Months 6-12)

- Learned reranking
- More model providers
- Advanced evaluation/reputation
- Provider payouts
- Enterprise controls
- Dedicated execution pools
- Advanced cost optimization
- Expanded integration catalog

22. Suggested Engineering Team

Role	Initial scope
Product / founder	Product decisions, marketplace economics, customer discovery
Frontend engineer	Next.js, TypeScript, UI system, realtime UX
Backend engineer	NestJS, PostgreSQL, billing, auth, APIs
AI engineer	FastAPI, agent runtime, RAG, model routing, evaluations
Platform engineer	Docker, Kubernetes, Terraform, CI/CD, observability

Role	Initial scope
QA / automation	E2E, contracts, load/resilience/security automation

23. Production Launch Checklist

- All production data paths have tenant authorization and RLS coverage where applicable.
- Backups are configured and restore has been tested.
- Temporal workflows are idempotent and resilient to worker restart.
- LLM provider failover is tested.
- Qdrant tenant filters are enforced and tested.
- Neo4j graph projections are recoverable from PostgreSQL source records.
- Secrets are in managed secret storage, not repository/config maps.
- Kubernetes readiness/liveness probes and autoscaling are configured.
- CI blocks vulnerable dependencies above agreed thresholds.
- Payment webhooks are authenticated and idempotent.
- Usage metering is reconciled against provider costs.
- Agent kill switch is tested.
- Incident response contacts and runbooks are documented.
- Key SLOs and alerts are visible in production dashboards.
- Customer-facing Terms, Privacy, billing, refunds, and data-use policies are approved.

24. Architecture Principles to Keep

1. PostgreSQL is the source of truth

Do not split business truth across multiple databases. Graph and vector systems are purpose-built projections/services.

2. Temporal owns durable workflow state

A database row is not a substitute for durable orchestration of long-running or retry-heavy agent work.

3. Security is enforced at multiple layers

Identity, application authorization, database RLS, vector tenant filters, scoped credentials, and network boundaries should work together.

4. Agents are versioned products

Every production agent must have a version, test/evaluation state, capability specification, pricing rules, and

rollback path.

5. Observability is part of the product

Customers and operators need reliable visibility into execution status, outcome, cost, and failures.

25. Final Target

```
AgentKart
= Global marketplace + transaction layer + execution platform

Frontend:    Next.js / React / TypeScript
Core platform: NestJS / TypeScript
AI platform:  FastAPI / Python
Workflow:    Temporal
Relational:  PostgreSQL + RLS
Graph:       Neo4j
Vector:      Qdrant
Cache/realtime: Redis
Artifacts:   S3
Infra:       Docker + Kubernetes
Cloud:       AWS or GCP
IaC:         Terraform
Observability: OpenTelemetry + Prometheus + Grafana + Sentry
```

This is the baseline to implement. The architecture is intentionally strong enough for enterprise use, while the product roadmap controls which capabilities are exposed to customers first. New subsystems should be added only when they have a defined responsibility and operational owner.